Further Lean Proof Simplifications

Integer geometry, fixed-index Pell estimates, color moments, and canonical Wang rows

Research and checked prototypes for Vladimir Reshetnikov

26 October 2026

Abstract

This report proposes four further directions for simplifying the maintained Lean development of Diophantine consistency. Its baseline is deliberately counterfactual: every simplification in the two excluded collections is treated as available, whether adopted upstream or not. The initial eighteen-packet audit was extended to four newly added packets at the final exclusion cutoff. The remaining opportunities concern different proof obligations: integer-period normalization, a fixed Pell index instead of a variable-exponent argument, sufficient color statistics for packed coefficients, and a canonical recognition theorem connecting an arbitrary allowed Wang row to its existing explicit construction. Exact local proof replacements are distinguished from reusable infrastructure and architectural proposals. No global line-saving or build-speed claim is extrapolated from a small prototype.

Repository pin. `553a5f7a463c`. The complete SHA is recorded in the evidence files. The repository specifies Lean `v4.32.0` and Mathlib commit `81a5d257c8e410db227a6665ed08f64fea08e997`.

Recommendation. Start with the helical period proof and the index-six Pell lemma. Pilot exact recognition of operating rows before extending it to other row families. Develop the moment method around its four concrete signal-mass clients; do not count the already-proposed generic chunk assembler as a new saving.

ID	Targeted simplification	Evidence in this packet
P1	Bilateral integer induction for a helical orbit and seam invariant	Exact helix signature and generic period-one helper checked.
P2	Variable-index power comparison above a fixed threshold	Exact original signature checked; no new helper.
P3	Repeated evaluation of bit-packed signal coefficients	Symbolic design; arithmetic endpoints checked in Python.
P4	Separate tile construction and recognition calculations	Source-reviewed recognition design; Lean integration unverified.

Contents

1 What was reviewed, and what counts as additional **3**
1.1 Important exclusions . . . 4
1.2 Validation vocabulary . . . 4

2 P1: normalize helical orbits using the algebra of periods **4**
2.1 Target and existing argument . . . 4
2.2 The replacement . . . 4
2.3 A second client: seams at every integer time . . . 5
2.4 Evidence and cost . . . 5

3 P2: replace the index-six Pell threshold by a fixed anchor **6**
3.1 Target and counterfactual baseline . . . 6
3.2 The minimal fixed-index proof . . . 6
3.3 Why an exact polynomial is optional . . . 6
3.4 Lean interface and cost . . . 6

4 P3: certify signal masses through sufficient color statistics **7**
4.1 Why another chunking proposal would be redundant . . . 7
4.2 The affine factorization . . . 7
4.3 Signal-family formula . . . 7
4.4 A concrete pilot and four consumers . . . 8
4.5 Cost discipline . . . 9

5 P4: recognize operating rows by their canonical form **9**
5.1 Retain the whole receiver tile in the recognition proof . . . 9
5.2 A stronger recognition route: local tile determinism . . . 9
5.3 Noncircular dependency plan . . . 11

6 Implementation priorities and acceptance criteria **11**

7 Deliverables and reproduction **12**

Source and evidence references **12**

1 What was reviewed, and what counts as additional

The source pin contains 445 Lean files in the maintained `DiophantineConsistency` module directory. This count includes checks and audits there; it is not a count of independent mathematical modules. The review concentrated on the exact source targets identified below and their proof dependencies, rather than claiming a line-by-line review of all 445 files.

The initial exclusion corpus consists of the six packets `2026-10-26a` through `2026-10-26f`, their adoption record and surrounding index, all twelve incoming ZIPs, and the incoming README. There are 323 files in the placed packets and 2,405 members in the incoming archives; 2,084 textual archive members were extracted. The current 336 repository files comprising the placed collection, the twelve ZIPs and the incoming README were checked against the pinned Git blob identities with zero mismatches. The ZIPs total 11,021,773 bytes. No nested ZIPs or additional DOCX-only reports were found.

The audit indexed 35 principal manuscript or section units and 590 headings. It also reviewed supplementary technical notes, prior detailed exclusion catalogs, candidate-code inventories, and searches targeted to each new proposal. Unchanged bundled source snapshots were treated as evidence about implementations, not as thousands of separate proposals. The audit does not claim semantic verification of every snapshot or every historical paper linked remotely from the incoming index.

The operative test is mechanism, not terminology. Renaming a generic fold, specializing a previously proposed transport lemma, or implementing an earlier suggestion more neatly does not make an additional idea. Each section therefore identifies the closest prior proposal and the exact new obligation it removes. A proposed benefit must remain after the older infrastructure is assumed installed.

Final exclusion cutoff. The repository advanced during review. The exclusion audit was therefore extended to `7def5e00ef35`, containing twenty-one placed packets (a–u) and one incoming ZIP. Twelve previously incoming packets had been relocated into g–r. The genuinely additional material is s, t, u, and `ProveIt_Lean_Simplification_Round3_2026-10-26.zip`. The supplementary audit and exact identities are in `audit/FRESHNESS.md`.

This final check removed two initially selected directions: centered integer induction and shared-edge row construction. Both are already present in the new material. They are discussed only as exclusions or assumed infrastructure, and their prototypes, patches and measured reductions are not included as recommendations. All source targets retained here have the same contents at the source pin and the final exclusion cutoff; the broader repository does have unrelated source changes.

The accompanying `audit/EXCLUSIONS.md` and `audit/exclusions.json` contain 131 overlapping coverage records. These initial records comprise 75 earlier coverage families, attributed and cross-checked, followed by 56 refinements and notes-only exclusions. The final-cutoff addendum records the additional exclusions separately. They are not 131 disjoint inventions.

Delivery recheck. At `45ac0dfc366f`, Round3 has been relocated to packet v and the incoming directory contains only its README. The substantive twenty-two-packet corpus is unchanged: a–u have identical Git tree identities, and every Round3 member is accounted for by identical content, one newline-only normalization, or the placement's recorded omission manifest. Sixteen retained source/dependency files also have unchanged Git blobs at this delivery revision. See `audit/DELIVERY_CHECK.md` and `audit/delivery-source-blobs.json`.

1.1 Important exclusions

The following mechanisms are already covered and are not recommendations of this report: generic scanner/fixed-point iteration; successful decoder branches and fused decoder invariants; assignment and renaming transport; generic circuit folds and metadata certificates; sum-of-squares packaging; finite support classification; cyclic carry uniqueness and explicit extremal carries; Pell base monotonicity and small-ordinate fundamental-unit criteria; finite boundary-color fibers; aligned-cell coordinate equivalences; and physical selector coherence.

Several less visible exclusions matter. Technical notes already propose cyclic packing through a monoid-algebra homomorphism, support-restricted evaluation/frame rules, arbitrary invertible linear transformations of residual rows, shared guarded constructor witnesses, and natural-semiring evaluator interfaces. None is reintroduced here.

1.2 Validation vocabulary

Exact local replacement. The theorem statement is preserved and its new proof is checked in Lean. A standalone exact-signature check is distinguished from rebuilding the whole production module.
Checked helper or pilot. A new mathematical core or a concrete production-data example compiles. This does not certify an unimplemented migration of all clients.
Architectural proposal. The report gives a mathematical construction and a noncircular dependency plan. Estimated opportunity is not reported as achieved source saving.

Two retained standalone Lean files completed successfully: the exact helical theorem with a generic period-one helper, and the exact Pell threshold theorem. The seam adaptation is supplied as a source patch supported by the checked generic helper; its full production module was not rebuilt. The symbolic signal-moment adapter timed out without diagnostics and is explicitly unvalidated. The canonical-row and edge-recognition files are proposed integrations, not a completed Lean check. An optional full production dependency rebuild was stopped before completion; no whole-project build, runtime improvement or structural net line saving is claimed.

2 P1: normalize helical orbits using the algebra of periods

2.1 Target and existing argument

The theorem `row_zero_of_helix` in `WangTiles942HelicalQuotient`, lines 22–40 assumes

$$T(x + w, t - 1) = T(x, t)$$

for every integer pair and proves

$$T(x, t) = T(x + wt, 0).$$

Its present proof performs induction over both positive and negative integer times, generalizes the horizontal coordinate, and normalizes a ring identity in each branch. The argument is valid but duplicates a closure property that the library already proves for periods in arbitrary additive groups.

2.2 The replacement

Let $F : \mathbb{Z}^2 \to \alpha$ be the uncurried function $F(x, t) = T(x, t)$. The hypothesis says that $(w, -1)$ is a period of F. Every integer multiple of a period is a period. Applying the multiple $t(w, -1)$ at (x, t)

gives

$$F\big((x,t)+t(w,-1)\big)=F(x,t),\qquad (x,t)+t(w,-1)=(x+tw,0).$$

This proves the target after reversing the equality. The reasoning includes negative t, zero w, and negative w; it needs no finite modulus, positivity, division, or coprimality assumption.

The replacement proof is short enough to show in full, with the theorem's existing variables in scope:

```
have hp : Function.Periodic (Function.uncurry T) (w, -1) := by
  intro p
  simpa only [Function.uncurry, Prod.fst_add, Prod.snd_add,
    sub_eq_add_neg] using hhelix p.1 p.2
simpa [Function.uncurry, zsmul_eq_mul, mul_comm]
  using (hp.zsmul t (x,t)).symm
```

The API is `Function.Periodic.zsmul` at the pinned Mathlib revision. This is a direct use of an existing theorem, not a proposed new quotient framework.

2.3 A second client: seams at every integer time

In `WangTiles942SeamColumns`, `seam_iff_zero`, the already-proved one-step equivalence

$$\mathrm{SeamAt}(x,t+1)\iff \mathrm{SeamAt}(x,t)$$

is extended to all integer times with another bilateral induction. Turn this equivalence into equality of propositions using `propext`; the predicate as a function of time has period one. The library's `Periodic.int_mul_eq` immediately gives equality with its value at zero.

This adds no period assumption: period one is derived from the existing successor equivalence. It also does not assert that the tile itself is constant. Only seam status is constant, exactly as in the old theorem.

2.4 Evidence and cost

The exact-signature `HelicalPeriod.lean` compiles with Lean 4.32.0 and pinned Mathlib. The two proposed production theorem blocks change from 19 to 8 lines and from 8 to 4 lines. The complete two-file candidate changes from 253 to 238 physical lines, a 15-line reduction, with no new production helper. Of these 15 lines, 11 belong to the exact checked helix replacement and 4 to the seam adaptation whose production integration remains unchecked. These are source-patch measurements, not a full-module build certificate.

The standard propositional/extensional axioms used by a helper must be recorded rather than silently described as axiom-free. In particular, the seam adaptation explicitly uses `propext`. The supplied validation evidence states the scope of each axiom printout.

Difference from previous proposals. Earlier cell-cycle propagation and full-modulus geometric normal forms concern finite aligned cells and residue arithmetic. This proof instead removes an external signed-time induction by using one vector period in $\mathbb{Z}^2$. No prior matching proposal was found in the inspected corpus.

3 P2: replace the index-six Pell threshold by a fixed anchor

3.1 Target and counterfactual baseline

The target is `large_of_six_le` in `NativePellBootstrap`, the index-six size lemma. It proves, for $A > 1$, $6 \le p$, and $c = y_A(p)$,

$$A(A^2-1)^2 < c.$$

The current argument uses variable powers and lower growth bounds for the Pell sequence. Earlier proposals already simplify scale-dependent estimates and compare different Pell bases. Their patched versions retain this particular threshold argument. The new reduction keeps the base fixed and uses only a fixed-index comparison at six.

3.2 The minimal fixed-index proof

The library already supplies the lower bound $A^n \le x_A(n)$, the positive recurrence

$$y_A(n+1) = x_A(n) + Ay_A(n),$$

and strict monotonicity of y_A in its index. Their fixed-index consequences give the chain

$$A(A^2-1)^2 < A^5 \le x_A(5) \le y_A(6) \le y_A(p) = c. \tag{1}$$

The first inequality uses $A > 1$, hence $0 < A$ and $A^2 - 1 < A^2$, and the strict monotonicity of the square on natural numbers. The second is `xn_ge_a_pow` at the literal index five. The third is the positive recurrence at five. The fourth uses `(strictMono_y hA).monotone hp`.

Only the fixed exponents two and five occur in the proof. The old intermediate quantity $(2A-1)^{p-1}$, the subtraction $p-1$, its restoration to p, and the comparison between variable powers disappear. The index p is handled once, by monotonicity.

3.3 Why an exact polynomial is optional

For mathematical context, put $D = A^2 - 1$. Expanding $(A+u)^6$ under $u^2 = D$ gives

$$y_A(6) = 6A^5 + 20A^3D + 6AD^2.$$

This positive-coefficient expression also proves the threshold immediately, since $AD^2 > 0$. It is preferable to the subtraction-heavy equivalent expression $32A^5 - 32A^3 + 6A$ if one actually needs an exact value.

However, the production target only asks for an inequality. The shorter implementation in this packet uses (1); it does not add an exact-polynomial helper or make clients unfold six recurrence steps. The exact expansion is an explanatory mathematical identity here, not an additional claimed compiled deliverable.

3.4 Lean interface and cost

The supplied `PellAnchorSmall.lean` preserves the original `large_of_six_le` signature. Its proof uses `xn_ge_a_pow hA 5`, the definitional successor equation through `Nat.le_add_right`, and index monotonicity. No new theorem is needed in the production import graph. This is intentionally a small deletion of variable-exponent administration, not a general library of Pell threshold estimates.

The exact-signature `PellAnchorSmall.lean` compiled successfully (exit 0), importing only `Mathlib.NumberTheory.PellMatiyasevic`. It neither imports the project theorem being replaced nor adds a helper declaration. The production patch changes the declaration from 15 to 11 physical lines and its proof body from 13 to 9 lines: 4 lines removed. The patch applies cleanly to the pinned checkout. The full `NativePellBootstrap` module was not rebuilt. The axiom printout is `propext`, `Classical.choice`, `Quot.sound`; no stronger axiom-preservation claim is made.

Difference from previous proposals. Base monotonicity transports $y_A(p)$ between different A. A small-ordinate fundamental-unit criterion identifies the least positive unit. Neither replaces this arbitrary-index growth proof by a fixed anchor followed by index monotonicity. That distinction remains after those earlier improvements are assumed applied.

4 P3: certify signal masses through sufficient color statistics

4.1 Why another chunking proposal would be redundant

`CurrentKernel2011`, `selector_mass` and the signal-family masses currently certifies large sums of packed selector weights. A generic checked chunk assembler has already been proposed. Its adoption must be assumed here. Deleting the visible ladder of weighted chunk lemmas is therefore *not* a saving attributable to this report.

The additional observation is mathematical: the packed selector weight is affine in four color coordinates. Consequently a whole family's mass is determined by its cardinality and four small color sums. For the signal families those statistics have symbolic formulas, reusable for both full and cutoff families. This changes what must be certified, rather than hiding the same weighted computation behind another tactic.

4.2 The affine factorization

For a tile $t = (w, n, e, s)$, the actual weight satisfies

$$\mathrm{weight}(t) = 1025 + 1024w + 262144n + 134217728e + 34359738368s.$$

For a finite list of tiles L, define its five moments

$$(m, W, N, E, S) = \left(|L|, \sum_L w, \sum_L n, \sum_L e, \sum_L s\right).$$

Then

$$\sum_{t \in L} \mathrm{weight}(t) = 1025m + 1024W + 262144N + 134217728E + 34359738368S. \tag{2}$$

This is a list-additivity theorem proved once. It is a theorem about actual selector weights, not an unverified external numeric calculation.

4.3 Signal-family formula

Let $d \in \{0, 1\}$ be the movement direction and let S_0 be its actual target set, including the halting target when present. Write

$$n = |S_0|, \qquad Q = \sum_{q \in S_0} q, \qquad R = \sum_{q \in S_0} \mathrm{rail}(d, q).$$

There are two receiver tiles per target, one for each tape bit $b = 0, 1$. Their colors are

$$\big(d\,\mathrm{rail}(d,q),\ b,\ (1-d)\,\mathrm{rail}(d,q),\ 2+2q+b\big).$$

Summing the two values of b first gives

$$(m, W, N, E, S) = \big(2n,\ 2dR,\ n,\ 2(1-d)R,\ 4Q+5n\big). \tag{3}$$

Substitution into (2) gives

$$\mathrm{mass}_d(S_0) = 171798956034\,n + 137438953472\,Q + \begin{cases} 2048R, & d=1, \\ 268435456R, & d=0. \end{cases} \tag{4}$$

For the actual rank-based rails, a generic rank-sum identity can further replace R by

$$R = c_d n + \frac{n(n-1)}{2},$$

where c_d is the direction-specific rail offset: $c_1 = 6$ and $c_0 = 205$. The integrality of the triangular term should be proved through the finite sum of ranks, avoiding ad hoc natural-division rewrites.

4.4 A concrete pilot and four consumers

For the full right-signal family, the proposed moment certificate is

$$(m, W, N, E, S) = (398, 41790, 199, 0, 91723),$$

which reconstructs the existing mass

$$3151578377695630.$$

The numeric pilot reconstructs the actual source data in Python. The separate proposed Lean adapter imports the production `WangTiles942Core`; it does not redefine the tile type or selector-weight function. That adapter did not complete its compiler check, so the numeric equality above is not presented as a certified new Lean theorem.

There are four concrete consumers: the full right/left masses in `CurrentKernel2011`, and the cutoff right/left masses in `CurrentSelectorTail2011`, `right_low_mass` and `left_low_mass`. The cutoff $2 + 2q + b \le 45$, with $b \in \{0, 1\}$, selects exactly $q \le 21$. This is an initial segment of the target order, so its global ranks agree with its ranks in that segment. After the proposed rank-sum lemma is installed, the same symbolic formulas apply and only the target count and target sum change. Until then, the rail sum remains a third statistic to certify.

The included Python computation reproduces all four existing signal-family endpoints, and the formulas were checked algebraically. The proposed `SignalMomentsCore.lean` compiler attempt timed out after 300 seconds without diagnostics; this is an unvalidated design artifact, not a proof success or a demonstrated proof error. The rail-rank sum, initial-segment adapter, and complete four-client migration remain implementation work. No achieved source saving or Lean performance improvement is claimed.

4.5 Cost discipline

The reusable implementation must include the bit-pair formula, enumeration bridge, rail-rank sum, and cutoff equivalence in its cost. Four closed moment certificates in place of one already-short weighted certificate could make the source longer. The credible benefit comes from symbolic sharing across full and cutoff families and reuse of the same moments if packing weights change. No faster kernel checking is claimed without an equal-scope measurement.

Difference from previous proposals. Checked chunk assembly evaluates weighted data, and packed-field descriptors describe layouts. This proposal proves a sufficient-statistics factorization and symbolic signal-family moments, so, once the rank bridge is installed, the remaining authenticated data are target counts and sums. It is narrower than a generic linear-algebra framework and does not rely on a new numerical encoding.

5 P4: recognize operating rows by their canonical form

The largest structural opportunity retained here concerns recognition. The earlier packets already propose constructing rows from shared edge values, including the operating-step pulse and the live-state adapter. Those ideas are part of the assumed baseline and receive no novelty or line-saving credit here.

The additional invariant goes in the other direction: an arbitrary allowed, horizontally matching operating row which encodes a given configuration is the already-defined explicit `stepTile` row on the strip interior. Construction semantics can then interpret that row once, rather than being duplicated in the soundness proof. The literal tiles, selector encoding and 942-element language are unchanged.

5.1 Retain the whole receiver tile in the recognition proof

On the recognition side, `WangTiles942RowStep`, `receiver_south` already obtains an equality identifying the complete receiver tile in each direction branch, then immediately applies `congrArg Tile.south`. The additional invariant is to keep that equality as `receiver_tile`.

Together with the existing exact `head_transition` and `far_pass` theorems, it gives the canonical-row statement

$$T(x,t) = \mathrm{stepTile}(\mathrm{cfg}, q, x) \qquad \text{for every interior position } -L < x < R. \tag{5}$$

The proof partitions the positions once. At the old head use `head_transition`; at the new head use `receiver_tile`; everywhere else use `far_pass`. The moved-head inequality rules out overlap.

Now the soundness statement becomes a projection:

$$\mathrm{south}(T(x,t)) = \mathrm{south}(\mathrm{stepTile}(\mathrm{cfg}, q, x)) = \mathrm{color}(\mathrm{step}(\mathrm{cfg}), x).$$

It reuses the construction-side semantics instead of redoing the machine's color case analysis in `south_step`.

5.2 A stronger recognition route: local tile determinism

The minimal route above preserves the old three-way position split. A more ambitious implementation can share the classification work itself. For an allowed row, first recover its east-edge function from the

head tile. With $h = \text{cfg.head}$, the candidate invariant on strip edges $-L \le x < R$ is

$$E_T(x) := (T(x,t)).\text{east} = \begin{cases} (T(h,t)).\text{east}, & x = h, \\ (T(h,t)).\text{west}, & x + 1 = h, \\ 0, & \text{otherwise.} \end{cases}$$

The second case is horizontal matching. In the third case neither endpoint carries a head. Prove one boundary-aware adjacent-pair lemma: both interior endpoints are plain, or one is the strip wall. The existing literal boundary-color classification then forces their shared edge to zero. State the lemma with its exact wall hypotheses; a theorem about two interior cells alone does not cover the wall-adjacent edge.

The remaining local fact is that an allowed tile is determined by its west, north and east colors once the exact reset/finish exceptions are excluded. The independently reconstructed literal data give the following complete ambiguity table:

West–north–east key	Possible south colors	Cause
$(0, n, 0)$, $n \in \{0, 1, 432, 433\}$	n or 438	pass or finish
$(1, 438, 1)$	$434, 435, 436$	reset alternatives

There are 942 distinct tiles and 936 distinct west–north–east keys. Exactly five keys are ambiguous, accounting for seven unequal unordered tile pairs. Requiring north $\neq 438$ and south $\neq 438$ leaves 935 tiles with 935 distinct keys. Thus the proposed local theorem is:

$$\begin{gathered} \text{Allowed}(a),\ \text{Allowed}(b), \qquad (a.w, a.n, a.e) = (b.w, b.n, b.e), \\ a.n \neq 438, \quad a.s \neq 438, \quad b.s \neq 438 \quad \Longrightarrow \quad a = b. \end{gathered}$$

The common north equality supplies $b.n \neq 438$. The non-finish premise is essential, not merely a convenient tactic condition. The operating-row invariant excludes reset inputs; the independent `no_south438` theorem excludes finish outputs. Halting receiver tiles remain admitted: entering target 215 is not the excluded finish transition.

Compare the arbitrary row and the canonical `stepTile` row using their equal head tile, equal north colors, the edge-shape invariant, and this local determinism theorem. Horizontal matching supplies the west-color equality from the preceding east edge. This offers a way to replace both `far_west`/`far_east` analyses and the direction-specific receiver analysis with one shared recognition argument. It should be tested against the existing body and helper costs before any saving is quoted.

The candidate boundary-aware edge proof is included as `EdgeRecognition.lean`; it too remains uncompiled. The 942-tile ambiguity calculation is supplied as `scripts/check_tile_determinism.py` and `validation/tile-determinism.json`. It verifies the authenticated action, target and rail data before enumerating the tiles. This is reproducible finite arithmetic evidence, *not* a new Lean proof of the local theorem. Earlier boundary-fiber machinery may be used to prove that theorem; the new proposal is its composition with edge recognition and canonical-row soundness, not another finite-classification technique.

5.3 Noncircular dependency plan

The required order is:

1. Keep the local language/signal classification and machine operational semantics below both row directions.
2. Use the explicit `stepTile` realization and its independently proved color semantics. The previous shared-edge construction can supply this layer if already installed.
3. Strengthen `receiver_south` to `receiver_tile` using its existing direction-local proofs. Keep the former name as a projection if clients need it.
4. Combine that whole-tile receiver fact with `head_transition` and `far_pass` to prove (5).
5. Derive `south_step` and then `row_step`. Do not use either theorem while proving the canonical-row equality.

The `far_pass` argument still needs exclusion of finish tiles, obtained through the existing `no_south438` theorem. It is not valid to drop temporal matching, persistent-wall assumptions, or the no-finish derivation merely because a canonical row has been introduced. The equality is only asserted on the strip interior; wall/seam tiles have different roles.

The supplied `CanonicalOperatingRow.lean` retains the complete receiver equality, states the interior canonical-row theorem, derives the south-color theorem by the construction API, and gives an immediate row-uniqueness corollary. The dependency plan has been source-reviewed, but the file was not compiled against a rebuilt production import graph. The canonical theorem still partitions old head, moved head and other positions; moving that partition is not by itself a reduction. The stronger API and reuse of color semantics are the benefit to test. Charge the additional statements and any compatibility wrappers before claiming net shortening.

Difference from previous proposals. Boundary fibers classify which literal tiles have a color. Selector coherence proves that two decoders choose the same tile index for one word. The new interface identifies the complete operating row from its upper configuration and couples soundness to the existing explicit construction. The earlier shared-edge mechanism generates a matching row from chosen data; it does not prove that every admissible operating row equals that row. This converse recognition obligation is the proposed addition.

6 Implementation priorities and acceptance criteria

Order	Change	Acceptance criterion
1	P1 period proofs and P2 index-six Pell anchor	Same public statements; exact local checks; axiom scope recorded; source patches apply to the source pin.
2	P4 canonical operating-row recognition	Strong receiver fact and canonical equality checked without `south_step`; construction and soundness clients share the semantic theorem.
3	P3 symbolic signal moments	Full and cutoff rank/count/sum bridges included; compare against the already-shortened chunk baseline.

The stages are small enough to review independently. No full-program size certificate should be changed

by these proof-only local replacements. For the structural changes, preserve the existing literal definitions and initially add equality theorems and corollaries. If a later refactor changes a data-producing definition, its syntax-sensitive certificates require their own review; semantic equivalence alone does not establish a literal node count.

Source line counts are useful only when the measurement boundary is explicit. The report counts entire affected theorem blocks or entire files where stated. It does not add together overlapping counts, claim that new named lemmas are free, or take credit for previous proposals. Elaboration time is especially environment-dependent here; the supplied logs record actual checks, but no speedup claim is made.

7 Deliverables and reproduction

- `report.tex`, `results.tex`, and `report.pdf`: the complete article and its compiled PDF.
- `lean/`: exact-signature replacements, mathematical helper prototypes, and structural prototypes whose status is recorded alongside them.
- `patches/`: reviewable source patches against the pinned repository; a patch's existence is not itself a whole-project build claim.
- `audit/`: exclusion catalog, corpus identities, precise source references, and peer-review notes.
- `validation/`: successful command outcomes, relevant compiler output, measured source sizes, and validation-scope metadata.
- `scripts/`: replay and source-audit helpers. Local cache paths used in this session are not required for normal replay in a built checkout.

Use the repository's pinned toolchain. Reproduction should first build the specified production imports, then run the prototype files through `lake env lean`. The supplied script documents exact module roots and keeps candidate outputs separate. A complete `lake build` of all repository targets is outside the validation claim of this packet.

Use `scripts/verify.py` in a checkout with the pinned Lean/Mathlib dependencies. Its default mode checks the two completed standalone files and verifies patch applicability without applying patches. The optional proposal mode also tries the unvalidated signal, canonical-row and edge-recognition files after their production imports have been built. The packet records exact inputs and outcomes; local cache locations from this review are not reproduction requirements.

Source and evidence references

[1] ProveIt commit `553a5f7a463c`, including `lean-toolchain` and `lake-manifest.json`.
[2] Placed proposal packets a–u and adoption record at the final cutoff.
[3] Incoming proposal archive and README at the final cutoff. The initial twelve archive identities are in `audit/corpus-summary.json`; final additions are recorded in `audit/FRESHNESS.md`.
[4] `WangTiles942HelicalQuotient`, `row_zero_of_helix`; `WangTiles942SeamColumns`, `seam_iff_zero`.
[5] `NativePellBootstrap`, `large_of_six_le`; Mathlib's positive Pell sequence and index monotonicity.
[6] `CurrentKernel2011`, signal-family and selector masses; `CurrentSelectorTail2011`, low signal masses; `WangTiles942Core`, literal selector weights and tile families.
[7] `WangTiles942StepConstruction`, explicit operating-row construction; `WangTiles942RowStep`, recognized row evolution; `WangTiles942Signals`, receiver classification.
[8] Mathlib period multiplication.